

Rajalakshmi Engineering College

Name: M.S Nanthana shree
Email: 241901064@rajalakshmi.edu.in
Roll no: 241901064
Phone: 9360753427
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 9_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

A teacher is filtering a list of words provided by students. Some words contain too many vowels, making them difficult for a spelling competition. The teacher decides to remove all words that contain more than two vowels.

Help the teacher to implement it using ArrayList.

Input Format

The first line contains an integer N, representing the number of words in the list.

The next N lines contain a string representing the words (one per line).

Output Format

The output consists of words that contain two or less than two vowels, printed in the same order they appeared in the input. Each word is printed on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1

sri

Output: sri

Answer

```
import java.util.ArrayList;
import java.util.Scanner;

// You are using Java
class VowelFilter {
    public boolean hasMoreThanTwoVowels(String word) {
        int count = 0;
        for (int i = 0; i < word.length(); i++) {
            char c = word.charAt(i);
            if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
                count++;
                if (count > 2) return true;
            }
        }
        return false;
    }

    public void filterWords(int N, Scanner sc) {
        ArrayList<String> words = new ArrayList<String>();
        for (int i = 0; i < N; i++) {
            words.add(sc.nextLine().trim());
        }

        for (String word : words) {
            if (!hasMoreThanTwoVowels(word)) {
                System.out.println(word);
            }
        }
    }
}
```

```

    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        sc.nextLine(); // consume newline

        VowelFilter vf = new VowelFilter();
        vf.filterWords(N, sc);
    }

```

```

    }

    public class Main {
        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            int n = sc.nextInt();
            sc.nextLine();
            VowelFilter.filterWords(n, sc);
            sc.close();
        }
    }

```

Status : Wrong

Marks : 0/10

2. Problem Statement

Rahul, a stock trader, wants to analyze the stock prices of a company over several days. For each day, he wants to determine the stock span, which is the number of consecutive days (including the current day) where the stock price is less than or equal to the price on that day.

The stock span helps him understand how long a stock has been continuously increasing or staying the same. You need to help Rahul by computing the stock span for each day using a Stack data structure efficiently.

Example:

Input:

7

100 80 60 70 60 75 85

Output:

1 1 1 2 1 4 6

Explanation:

For each day:

Day 1: Price = 100 Span = 1 (Only this day)
Day 2: Price = 80 Span = 1 (Only this day)
Day 3: Price = 60 Span = 1 (Only this day)
Day 4: Price = 70 Span = 2 (Includes today and previous day)
Day 5: Price = 60 Span = 1 (Only this day)
Day 6: Price = 75 Span = 4 (Includes today and previous three days)
Day 7: Price = 85 Span = 6 (Includes today and previous five days)

Input Format

The first line contains an integer n , the number of days.

The second line contains n space-separated integers $prices[i]$, where $prices[i]$ represents the stock price on the i -th day.

Output Format

The output prints n space-separated integers representing the stock span for each day.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

100 80 60 70 60 75 85

Output: 1 1 1 2 1 4 6

Answer

```
import java.util.*;
```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] prices = new int[n];
        for (int i = 0; i < n; i++) {
            prices[i] = sc.nextInt();
        }

        int[] span = new int[n];
        Stack<Integer> stack = new Stack<>(); // stores indices

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && prices[stack.peek()] <= prices[i]) {
                stack.pop();
            }

            if (stack.isEmpty()) {
                span[i] = i + 1;
            } else {
                span[i] = i - stack.peek();
            }

            stack.push(i);
        }

        for (int i = 0; i < n; i++) {
            System.out.print(span[i]);
            if (i != n - 1) System.out.print(" ");
        }
    }
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rahul is working on a list manipulation problem where he needs to reverse a specific subarray using a stack. Given an array and two indices l and r, he wants to reverse only the portion of the array from index l to r (both inclusive) while keeping the rest of the array unchanged.

Since Rahul wants to solve this problem efficiently, he decides to use a stack to reverse the subarray in $O(r - l)$ time.

Your task is to help Rahul by implementing this functionality.

Input Format

The first line contains an integer n , the size of the array.

The second line contains n space-separated integers $arr[i]$.

The third line contains two integers l and r , denoting the start and end indices of the subarray to reverse.

Note: The array follows 0-based indexing.

Output Format

The output prints the modified array after reversing the subarray between indices l and r .

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

1 2 3 4 5 6

1 4

Output: 1 5 4 3 2 6

Answer

```
import java.util.*;
```

```
public class Main {
```

```
import java.util.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```

int[] arr = new int[n];
for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}

int l = sc.nextInt();
int r = sc.nextInt();

Stack<Integer> stack = new Stack<>();
for (int i = l; i <= r; i++) {
    stack.push(arr[i]);
}

for (int i = l; i <= r; i++) {
    arr[i] = stack.pop();
}

for (int i = 0; i < n; i++) {
    System.out.print(arr[i]);
    if (i != n - 1) System.out.print(" ");
}
}
}

```

Status : Wrong

Marks : 0/10

4. Problem Statement

Sarah, a warehouse manager, is managing a list of product names in her store's inventory system. She needs to perform basic operations like adding (inserting) new products, removing products that are sold out or discontinued, displaying all the products in stock, and searching for a specific product in the inventory list.

Sarah's goal is to manage the inventory using a list of product names (strings). The system allows her to perform the following operations using ArrayList:

Insert a Product: Sarah adds a new product to the inventory. **Delete a**

Product: Sarah removes a product from the inventory when it's sold or discontinued. Display the Inventory: Sarah checks all the products currently available in the inventory. Search for a Product: Sarah searches for a specific product in the inventory to check if it's available.

Input Format

The input consists of multiple space-separated values representing different operations on a product list. Each operation follows a specific format:

- 1 <product_name> - Adds <product_name> to the product list.
- 2 <product_name> - Removes <product_name> from the product list if it exists.
- 3 - Print all products currently on the list.
- 4 <product_name> - Checks if <product_name> exists in the list.

Output Format

The output displays,

For (choice 1) prints, " <item> has been added to the list."

For (choice 2) prints, " <item> has been removed from the list."

For (choice 3) prints, "Items in the list:" followed by each item in the list on a new line, or "The list is empty." if the list is empty.

For (choice 4) prints, " <item> is found in the list." or " <item> not found in the list."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 apple 1 banana 2 apple 3 4 apple

Output: apple has been added to the list.

banana has been added to the list.

apple has been removed from the list.

Items in the list:

banana

apple not found in the list.

Answer

```
import java.util.ArrayList;
import java.util.Scanner;

import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<String> products = new ArrayList<>();
        while(sc.hasNext()) {
            int choice = sc.nextInt();
            switch(choice) {
                case 1: {
                    String item = sc.next();
                    products.add(item);
                    System.out.println(item + " has been added to the list.");
                    break;
                }
                case 2: {
                    String item = sc.next();
                    if(products.remove(item)) {
                        System.out.println(item + " has been removed from the list.");
                    }
                    break;
                }
                case 3: {
                    if(products.isEmpty()) {
                        System.out.println("The list is empty.");
                    } else {
                        System.out.println("Items in the list:");
                        for(String p : products) {
                            System.out.println(p);
                        }
                    }
                    break;
                }
                case 4: {
                    String item = sc.next();
                    if(products.contains(item)) {
                        System.out.println(item + " is found in the list.");
                    }
                }
            }
        }
    }
}
```

```

    } else {
        System.out.println(item + " not found in the list.");
    }
    break;
}
}
}
}
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<String> list = new ArrayList<>();

        String input = sc.nextLine();
        String[] commands = input.split(" ");
        int i = 0;
        while (i < commands.length) {
            int choice = Integer.parseInt(commands[i]);
            switch (choice) {
                case 1:
                    if (i + 1 < commands.length) {
                        StringListOperations.insertItem(list, commands[i + 1]);
                        i += 2;
                    } else {
                        System.out.println("No string provided for insertion.");
                        i++;
                    }
                    break;
                case 2:
                    if (i + 1 < commands.length) {
                        StringListOperations.deleteItem(list, commands[i + 1]);
                        i += 2;
                    } else {
                        System.out.println("No string provided for deletion.");
                        i++;
                    }
                    break;
                case 3:
                    StringListOperations.displayList(list);
                    i += 1;
                    break;
            }
        }
    }
}

```

```
case 4:
    if (i + 1 < commands.length) {
        StringListOperations.searchItem(list, commands[i + 1]);
        i += 2;
    } else {
        System.out.println("No string provided for searching.");
        i++;
    }
    break;
}
}
}
```

Status : Wrong

Marks : 0/10